

RIS Lab 2 — Spring 2026 Assignment 01

Yassin Youssef, Ali Ibrahim, Yassein Mahmoud

Task 1: Robot Visualization State

Task 1.1: Connect

After connecting to the Duckietown WIFI network, we established connection with our designated robot (Luna) by exporting the ROS master URI using command

export ROS_MASTER_URI=http://luna.local:11311

The connection was verified after by executing the command **rostopic list**. This returned a list of the active ROS topics that were published by the robot therefore confirming successful connection with ROS Master.

From the available topics 2 were selected for analysis:

- /luna/joy
 - sensor_msgs/Joy
- /luna/kinematics_node/velocity
 - duckietown_msgs/Twist2DStamped

The first topic provides joystick input data, while the second publishes the robot's planar velocity information.

Task 1.2: Visualize Signals

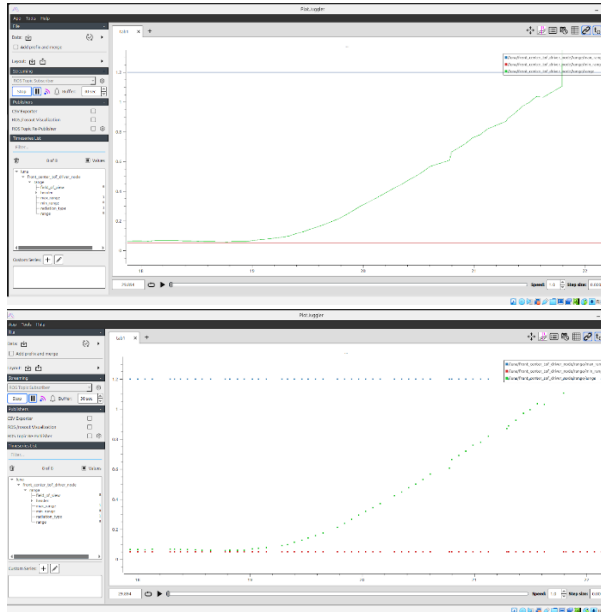


Figure 1.1 and 1.2: PlotJuggler visualization of the frontal range sensor data. The green curve represents the measured distance, while the red and blue lines indicate the minimum and maximum sensor range limits.

The frontal range sensor was visualized using Plotjuggler by subscribing to the topic **/luna/front_center_tof_driver_node/range**. From this topic, we then plotted 3 values: range, min_range and max_range.

The range value represents the distance between the sensor and the obstacle in front of the robot, while the min_range and max_range show the operational limits of the sensor.

We slowly moved the robot away from the obstacle resulting in a gradual increase in the range until it reached the maximum distance the robot can sense.

Task 1.3: Visualize the Robot State:

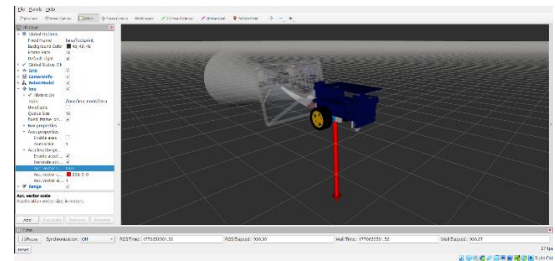


Figure 3: RViz visualization of the Duckiebot displaying the robot model, camera projection, IMU orientation, and frontal range sensor.

In this task, the robot state was visualized using RViz. The fixed frame was set to **/luna/footprint** to correctly reference the robot's coordinate system.

We then added other displays including Grid, RobotModel, CameraInfo, IMU, and the Range. The RobotModel display showed the 3D model of our Duckiebot, while the CameraInfo display visualized the projection of what's in front of the robot. Then the IMU displayed robot orientation and the Range displayed the limit of the frontal sensor.

This proved that the robot model and sensor were published correctly while being received in real time.

Task 2: Communication with the Robot

To enable communication between the laptop and the robot, the environment variable ROS_IP was exported using the IP address of the computer within the Duckietown network.

To command the robot wheels, we first identified the correct ROS topic and responsible for wheel control

by using the command: **rostopic list | grep -i wheel**
and then located the topic
/luna/wheels_driver_node/wheels_cmd. The
message type was verified by using the command

rostopic type

/luna/wheels_driver_node/wheels_cmd which
returned **duckietown_msgs/WheelsCmdStamped**.
The structure of the message was inspected using
rosmmsg show
duckietown_msgs/WheelsCmdStamped.

We then inspected the message field, the relevant
fields for wheel control are: **vel_left** and **vel_right**.

These fields represent the velocity of both the left and
right wheels, publishing positive values to both
wheels moves the robot forward while publishing zero
results in the robot coming to a stop.

A ROS package named **drive_forward** was created
containing a Python node that publishes wheel
commands. The node sends forward velocity
commands for 3 seconds and then explicitly
publishes a stop command before shutting down.

This package and code represents an open-loop
control since the robot moves for a fixed duration
without using feedback from sensors.

Before testing, the robot was lifted from the surface
to allow safe execution while the wheels spun freely.
After running the node, the robot successfully moved
forward for 3 seconds and then stopped, confirming
proper communication between the computer and
the Duckiebot.

Individual contributions:

**Yassin Youssef: Task 1.2 and Task 1.3 and lab
report**

Ali Ibrahim: Task 1.1 and Task 2

Yassein Mahmoud: Task 1.3 and Task 2